Rough : LAB-17 (3D plot)

Code Analysis

(Imports)

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import math as m
```

Calling :- Dataset :-

```
df = pd.read_csv ('transform.csv')
```

```
df['Label'] = df['Label'].map({'Blue':1, 'Red':0})
```

Do this operation on 'Label' colum

Select that Colum which has index as 'Label'

map that column to

Enter dictionary if Blue the set as 1 if Red the set as 0

```
X = df.iloc[:, :-1]
```

index lock — all rows — all colum Except last column

```
y = df['Label']#(Label's y as) take
```

* creation of feature maping function called transform() # Increase Dimendity to $R^2 \to R^3$ to resolve classification proble thus moving Feature from R^2 dimentional space to R^3 (dimentioned)

$$\phi : R^2 \longrightarrow R^3 \text{ defined by}$$

$$\boxed{\phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)}$$

```
def transform(X):
    X = X.value.tolist()   # Converting values to list
    newdf = []             # initializing Empty list
    for i in range(len(X))
        Var = ((X[i][0]**2), m.sqrt(X[i][1]*2[0][0]), (X[i][1]**2))
        newdf.append(Var)
    return pd.Dataframe(newdf)
```

X is pandas data frame they

Doing Same thing but by other efficient method

```
def transform_vectorized(X)
    X0 = X.iloc[:,0]
    X1 = X.iloc[:,1]

    C1 = X0**2
    C2 = (2 * X0 * X1)**0.5
    C3 = X1**2
return pd.Dataframe({0:C1, 1:C2, 2:C3})
```

sqrt Vaua Exponent

Using Numpy

```
def transfor_numpy(X):
    Vals = X.values
    X0 = vals[:,0]
    X1 = Vals[:,1]
new_data = np.colum_stack([
        X0**2,
        np.sqrt(2*X0,*X1)
        X0**2
    ])
return pd.Dataframe(new_data)
```

Solving for Cross checking & Practice

X1,	X2		1	2	3
1	13		1	18.384	169
1	18		1	25.45	324
2	9	→	4	18	81
3	6		9	18	36
6	3		36	18	9
9	2		81	18	4

$$(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

2D plot function

```
def plot_data (X1, X2, Y)
    plt.figure (figsize =(6,5))
    for label in np.unique(Y):
        plt.scatter (X1[Y== label]
                , X2[Y== label], label=label)

    plt.xlabel ("X1")
    plt.ylabel ("X2")
    plt.title (original Data(2D)")
    plt.legand ()
    plt.show()
```

---

improved version

```
def scatter_plot_2d (X, Y):
    x0 = X.iloc [:, 0].values
    x1 = X.iloc[:, 1].values

    plt.figure (figsize = (6,5))
    for label in np.unique(Y):
        mask = (Y==label) # select only
                            row belonging
                            to this label
    plt.scatter (x0[mask], x1[mask],
                label = f'class {label}', alpha
                = 0.7)

    plt.xlabel ('feature X0')
    plt.ylabel ('feature X1')
    plt.title ('2D feature Distribution')
    plt.legend()
    plt.grid (True, linestyle='--',
                alpha=0.5)

    plt.show
```

```
def scatter_plot_3d( transformed_df, y)
    fig = plt.figure (figsize = (6,5))
    ax = fig.add_subplot(111, projection='3d
    for label in np.unique(y):
        ax.scatter(
            transformed_data[Y=label,0],
            transformed_data [Y=label,1],
            transform_data_[Y=label, 2]
            label =label
        )

    xx, yy = np.meshgrid(
        np.linspace (min (transformed_
                    data[:,0)),
        max (transformed_data [:, 0))
        num = 10),

        np.linespace(min (transformed_data
                [:,1]
                max(transformed_data)
                [:,1]
                num 10)
        )

    zz = 3.00 - 0.5*xx - 4.5 * yy

    ax.plot_surface (xx, yy, zz,
                alpha = 0.3)
    ax.set_xlabel ("x1^2")
    ax.set_ylabel ("sqrt-2 x1x2")
    ax.set_zlabel ("x2^2")
    plt.legend()
    plt.show()
```